

Data Science Bootcamp

Take-home Exercise - Week 5

Submitted by - Meenakshi Madhu (mm14029)

Q1. <https://leetcode.com/problems/actors-and-directors-who-cooperated-at-least-three-times/>

Solution:

SQL:

```
SELECT actor_id, director_id
FROM ActorDirector
GROUP BY actor_id, director_id
HAVING COUNT(*) >= 3;
```

Pandas:

```
import pandas as pd
def actors_and_directors(actor_director: pd.DataFrame) ->
pd.DataFrame:
    result = (
        actor_director
        .groupby(['actor_id', 'director_id'])
        .size()
        .reset_index(name='count')
        .query('count >= 3')[['actor_id', 'director_id']]
    )
    return result
```

Q2. <https://leetcode.com/problems/fix-names-in-a-table/>

Solution:

SQL:

```
SELECT user_id, CONCAT(UPPER(LEFT(name, 1)), LOWER(SUBSTRING(name,
2))) AS name
FROM Users
ORDER BY user_id;
```

Pandas:

```
import pandas as pd
def fix_names(users: pd.DataFrame) -> pd.DataFrame:
    users['name'] = users['name'].str.capitalize()
    return users.sort_values('user_id')
```

Q3. <https://leetcode.com/problems/combine-two-tables/>

Solution:**SQL:**

```
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person as p
LEFT JOIN Address as a ON p.personId = a.personId;
```

Pandas:

```
import pandas as pd
def combine_two_tables(person: pd.DataFrame, address: pd.DataFrame) ->
pd.DataFrame:
    result = pd.merge(
        person, address,
        on='personId',
        how='left'
    )[['firstName', 'lastName', 'city', 'state']]
    return result
```

Q4. <https://leetcode.com/problems/second-highest-salary/>

Solution:**SQL:**

```
SELECT
    (SELECT DISTINCT salary FROM Employee
     ORDER BY salary DESC
     LIMIT 1 OFFSET 1
    ) AS SecondHighestSalary;
```

Pandas:

```
import pandas as pd
def second_highest_salary(employee: pd.DataFrame) -> pd.DataFrame:
```

```

    unique_salaries =
employee['salary'].drop_duplicates().sort_values(ascending=False).reset_index(drop=True)
    second_highest = unique_salaries[1] if len(unique_salaries) > 1
else None
    return pd.DataFrame({'SecondHighestSalary': [second_highest]})

```

Q5. <https://leetcode.com/problems/list-the-products-ordered-in-a-period/>

Solution:

SQL:

```

SELECT p.product_name, SUM(o.unit) AS unit
FROM Products p
JOIN Orders o ON p.product_id = o.product_id
WHERE YEAR(o.order_date) = 2020
AND MONTH(o.order_date) = 2
GROUP BY p.product_id, p.product_name
HAVING SUM(o.unit) >= 100;

```

Pandas:

```

import pandas as pd
def list_products(products: pd.DataFrame, orders: pd.DataFrame) ->
pd.DataFrame:
    orders['order_date'] = pd.to_datetime(orders['order_date'])
    feb_orders = orders[
        (orders['order_date'].dt.year == 2020) &
        (orders['order_date'].dt.month == 2)
    ]
    total_units = (
        feb_orders.groupby('product_id', as_index=False)['unit']
        .sum()
        .query('unit >= 100')
    )

    result = total_units.merge(products,
on='product_id')[['product_name', 'unit']]
    return result

```

Q6. <https://leetcode.com/problems/replace-employee-id-with-the-unique-identifier/>

Solution:**SQL:**

```
SELECT euni.unique_id, e.name
FROM Employees as e
LEFT JOIN EmployeeUNI as euni
ON e.id = euni.id;
```

Pandas:

```
import pandas as pd

def replace_employee_id(employees: pd.DataFrame, employee_uni:
pd.DataFrame) -> pd.DataFrame:
    result = pd.merge(
        employee_uni,
        employees,
        on='id',
        how='right'
    )[['unique_id', 'name']]

    return result
```

Q7. <https://leetcode.com/problems/game-play-analysis-iv/>**Solution:****SQL:**

```
SELECT
    ROUND(COUNT(DISTINCT a1.player_id) / COUNT(DISTINCT a.player_id),
2)
    AS fraction
FROM Activity a
LEFT JOIN Activity a1
    ON a.player_id = a1.player_id
    AND a1.event_date = DATE_ADD(a.event_date, INTERVAL 1 DAY)
WHERE a.event_date = (
    SELECT MIN(event_date)
    FROM Activity a2
    WHERE a2.player_id = a.player_id
);
```

Pandas:

```
import pandas as pd

def gameplay_analysis(activity: pd.DataFrame) -> pd.DataFrame:
    activity['event_date'] = pd.to_datetime(activity['event_date'])
    first_login =
activity.groupby('player_id')['event_date'].min().reset_index()
    first_login.columns = ['player_id', 'first_login']

    merged = pd.merge(activity, first_login, on='player_id',
how='left')

    next_day = merged[
        merged['event_date'] == merged['first_login'] +
pd.Timedelta(days=1)
    ][['player_id']].nunique()

    ### total players
    total = activity['player_id'].nunique()
    fraction = round(next_day / total, 2)

    return pd.DataFrame({'fraction': [fraction]})
```

Q8. <https://leetcode.com/problems/project-employees-i/>

Solution:**SQL:**

```
SELECT
    p.project_id,
    ROUND(AVG(e.experience_years), 2) AS average_years
FROM Project as p
LEFT JOIN Employee as e
ON p.employee_id = e.employee_id
GROUP BY p.project_id
```

Pandas:

```
import pandas as pd

def project_employees_i(project: pd.DataFrame, employee: pd.DataFrame)
-> pd.DataFrame:
```

```

result = (
    project.merge(employee, on='employee_id')
    .groupby('project_id', as_index=False)['experience_years']
    .mean()
    .round(2)
)
result.columns = ['project_id', 'average_years']
return result

```

Q9. <https://leetcode.com/problems/department-top-three-salaries/>

Solution:

SQL:

```

SELECT
    d.name AS Department,
    e.name AS Employee,
    e.salary AS Salary
FROM Employee as e
INNER JOIN Department as d
ON e.departmentId = d.id
WHERE (
    SELECT COUNT(DISTINCT e2.salary) FROM Employee as e2
    WHERE e2.departmentId = e.departmentId
    AND e2.salary > e.salary
) < 3
ORDER BY d.name, e.salary DESC

```

Pandas:

```

import pandas as pd
def top_three_salaries(employee: pd.DataFrame, department:
pd.DataFrame) -> pd.DataFrame:
    merged = pd.merge(
        employee,
        department,
        left_on='departmentId',
        right_on='id',
        suffixes=('_emp', '_dept')
    )

```

```
merged['salary_rank'] =
merged.groupby('departmentId')['salary'].rank(
    method='dense',
    ascending=False
)
result = merged[merged['salary_rank'] <= 3]
result = result[['name_dept', 'name_emp', 'salary']].copy()
result.columns = ['Department', 'Employee', 'Salary']
result = result.sort_values(['Department', 'Salary'],
ascending=[True, False])

return result
```